

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнил:
Алёшкин Дмитрий

Группа:
J3211

Проверил:
Добряков Д. И.

Санкт-Петербург

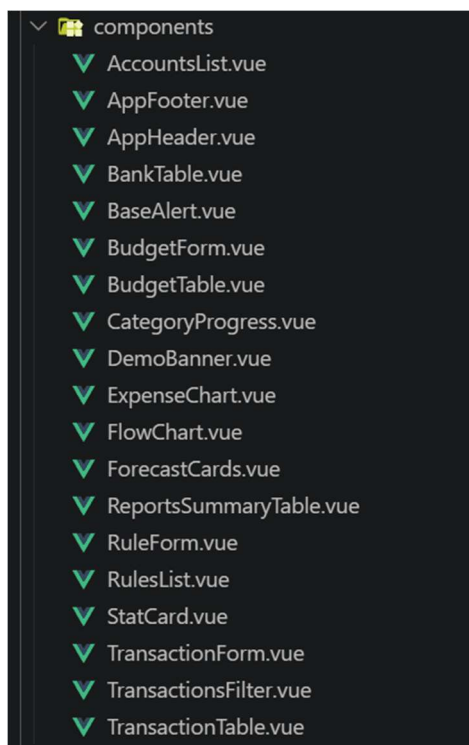
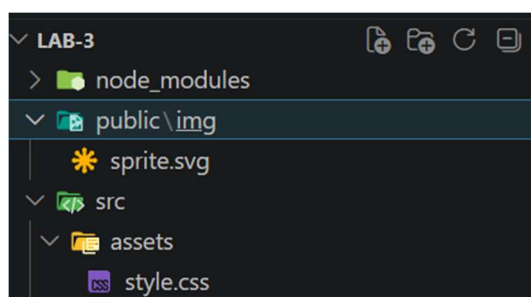
2026 г.

Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.JS.

Ход работы

В рамках лабораторной работы было выполнено преобразование ранее разработанного приложения FinanceManager с помощью фреймворка Vue.js 3. В предыдущей лабораторной работе приложение представляло собой набор отдельных HTML-страниц (dashboard, transactions, reports и т.д.), между которыми происходили переходы через стандартные ссылки. В новой версии вся логика была перенесена в Vue-приложение с единой точкой входа (main.js), а отображение страниц реализовано через динамическую подгрузку компонентов.



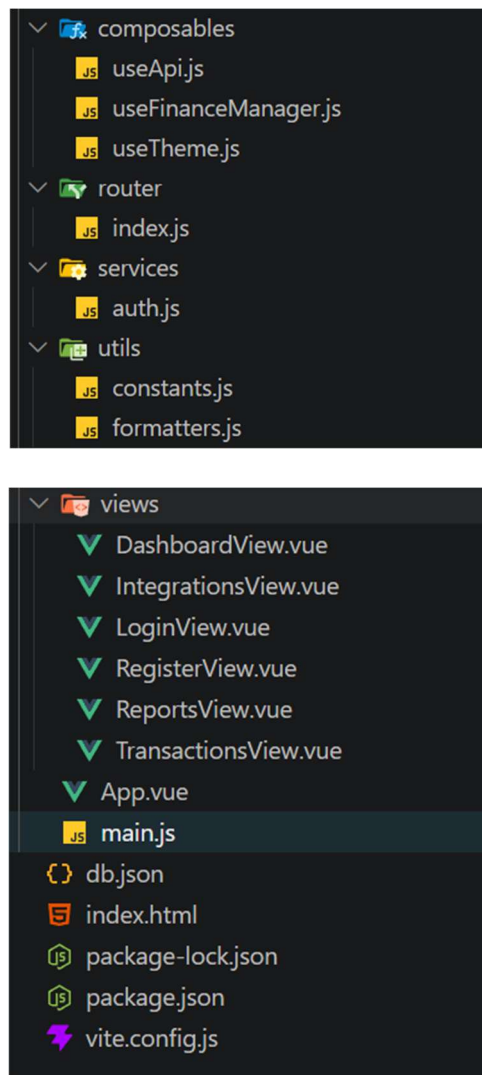
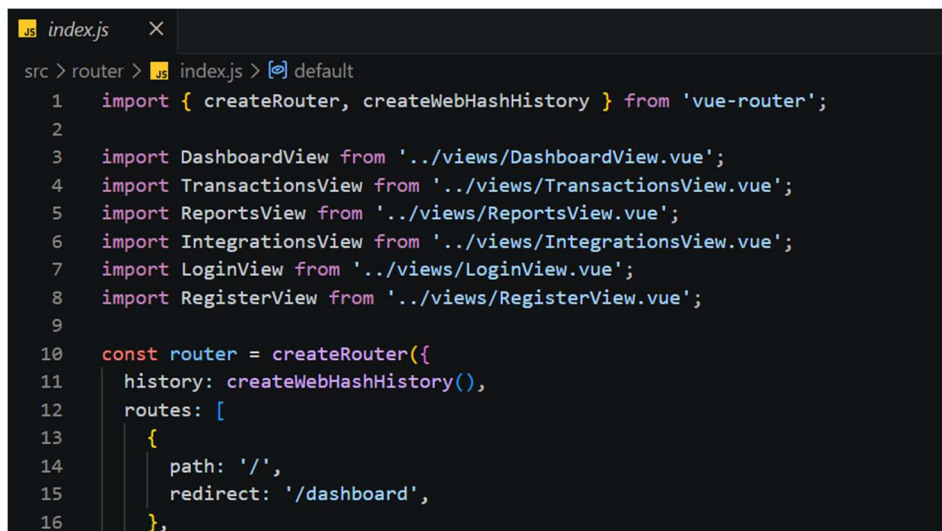


Рис. 1. Новая архитектура сайта

Для реализации навигации между разделами приложения был подключён Vue Router. Теперь переходы между страницами (Личный кабинет, Транзакции, Отчёты, Интеграции, Вход, Регистрация) осуществляются без перезагрузки страницы. Маршруты определены в отдельном файле `router/index.js`, а их отображение происходит через компонент `<router-view>`.



```

src > router > index.js > default
1  import { createRouter, createWebHashHistory } from 'vue-router';
2
3  import DashboardView from '../views/DashboardView.vue';
4  import TransactionsView from '../views/TransactionsView.vue';
5  import ReportsView from '../views/ReportsView.vue';
6  import IntegrationsView from '../views/IntegrationsView.vue';
7  import LoginView from '../views/LoginView.vue';
8  import RegisterView from '../views/RegisterView.vue';
9
10 const router = createRouter({
11   history: createWebHashHistory(),
12   routes: [
13     {
14       path: '/',
15       redirect: '/dashboard',
16     },
17   ],
18 });

```

Рис. 2. Фрагмент кода index.js, где определены маршруты

Интерфейс приложения был переработан с использованием компонентного подхода. В место монолитных HTML-страниц интерфейс был разбит на отдельные переиспользуемые компоненты:

- AppHeader — навигационная панель
- TransactionForm — форма добавления транзакции
- TransactionTable — таблица транзакций
- BudgetForm — форма бюджета
- ExpenseChart — диаграмма расходов
- FlowChart — график доходов и расходов

Такой подход позволяет разделить ответственность между частями приложения, упростить поддержку кода и повысить его читаемость. Все данные компоненты находятся в папке components.

Для взаимодействия с API была использована библиотека axios. В отличие от предыдущей версии, где использовались обычные функции и fetch, теперь все HTTP-запросы централизованы и выполняются через axios-инстанс. API реализовано с использованием json-server и файла db.json, который хранит пользователей, счета, транзакции и бюджеты.



```

src > composables > useApi.js > useApi
1  import axios from 'axios';
2  import { API_URL } from '../utils/constants.js';
3
4  const api = axios.create({
5    baseURL: API_URL,
6    headers: {
7      'Content-Type': 'application/json',
8    },
9  });
10
11 api.interceptors.response.use(
12   (response) => response,
13   (error) => {
14     if (!error.response) {
15       return Promise.reject(
16         new Error('Не удалось подключиться к mock API. Запустите json-server командой npm run api.'),
17       );
18     }
19   }

```

Рис. 3. Фрагмент кода, где происходит взаимодействие с API

Повторяющаяся бизнес-логика приложения была вынесена в composable-функции. Основной файл useFinanceManager.js содержит:

- загрузку данных пользователя
- работу с транзакциями
- работу с бюджетами
- расчёт доходов и расходов
- подготовку данных для графиков

Это позволило отделить бизнес-логику от компонентов и сделать код более структурированным.

```

useFinanceManager.js X
src > composables > useFinanceManager.js > useFinanceManager
115 export function useFinanceManager() {
168
169   async function login(email, password) {
170     await pingApi();
171
172     const normalizedEmail = email.trim().toLowerCase();
173     const { data } = await api.get('/users');
174
175     const user = data.find(
176       (item) =>
177         item.email?.trim().toLowerCase() === normalizedEmail &&
178         String(item.password ?? '') === String(password),
179     );
180
181     if (!user) {
182       throw new Error('Неверный email или пароль');
183     }
184
185     setStoredAuth(user);
186     currentUser.value = user;
187     isDemo.value = false;
188
189     await loadUserData();
190
191     return user;
192   }
193 }

```

Рис. 4. Фрагмент кода из useFinanceManager.js

В новой версии приложения визуализация данных (графики расходов и доходов) была интегрирована в компоненты Vue с использованием библиотеки Chart.js. Данные для графиков теперь автоматически обновляются при изменении транзакций благодаря реактивности Vue.



Рис. 5. Графики реализованные с помощью библиотеки Chart.js

Вывод

В ходе выполнения лабораторной работы я изучил основы разработки одностраничных приложений на Vue.js 3.

Были освоены:

- работа с Vue Router
- использование axios для работы с API
- компонентный подход
- использование Composition API и composables

Также я научился преобразовывать существующее приложение в SPA, разделять бизнес-логику и представление, а также организовывать структуру проекта. Полученные навыки позволяют разрабатывать более масштабируемые и удобные в поддержке веб-приложения.